Link Layer

Study-Ready Notes

Compiled by Andrew Photinakis

December 12, 2025

Contents

1 Introduction to the Link Layer (Section 6.1) 2
1.1 Conceptual Framework: The Node-to-Node Perspective . . . 2
1.1.1 Core Terminology and Abstraction . . . 2
1.2 The Transportation Analogy . . . 2
1.3 Services Provided by the Link Layer . . . 3
1.3.1 Framing . . . 3
1.3.2 Link Access (Medium Access Control, MAC) . . . 3
1.3.3 Reliable Delivery . . . 4
1.3.4 Error Detection and Correction . . . 4
1.4 Implementation of the Link Layer . . . 4
1.4.1 Location: The Network Adapter (NIC) . . . 4
1.4.2 Hardware–Software Interface . . . 5
1.4.3 Data Flow . . . 5
1.5 Summary and Reinforcement . . . 5

2 Error-Detection and -Correction Techniques (Section 6.2) 6
2.1 The Fundamental Problem: Bit Errors . . . 6
2.1.1 Concept Overview . . . 6
2.1.2 The Error-Detection Model . . . 6
2.1.3 Key Limitation: Undetected Errors . . . 7
2.2 Parity Checks . . . 7
2.2.1 Single-Bit Parity . . . 7
2.2.2 Two-Dimensional Parity . . . 7
2.3 Checksumming Methods . . . 8
2.3.1 Concept Overview . . . 8
2.3.2 The Internet Checksum (RFC 1071) . . . 8
2.4 Cyclic Redundancy Check (CRC) . . . 9
2.4.1 Concept Overview . . . 9
2.4.2 Mathematical Framework . . . 9
2.4.3 Sender Algorithm . . . 9

2.4.4 Receiver Algorithm . . . 9
2.4.5 Example: Polynomial Division . . . 9
2.4.6 Practical Power of CRC . . . 10
2.5 Summary and Reinforcement . . . 10
2.5.1 Key Terms . . . 10
2.5.2 Core Relationships . . . 11
2.5.3 Key Insights . . . 11

3 Multiple Access Links and Protocols (Section 6.3) 11
3.1 The Multiple Access Problem . . . 11
3.1.1 Concept Overview . . . 11
3.1.2 Collisions . . . 11
3.1.3 The Multiple Access Protocol . . . 11
3.2 Taxonomy of Multiple Access Protocols . . . 12
3.3 Channel Partitioning Protocols . . . 12
3.3.1 TDMA (Time Division Multiple Access) . . . 12
3.3.2 FDMA (Frequency Division Multiple Access) . . . 12
3.4 Random Access Protocols . . . 13
3.4.1 Slotted ALOHA . . . 13
3.4.2 Pure ALOHA (Unslotted) . . . 13
3.4.3 CSMA (Carrier Sense Multiple Access) . . . 13
3.4.4 CSMA/CD (Carrier Sense Multiple Access with Collision Detection) . 14
3.5 Taking-Turns Protocols . . . 14
3.5.1 Polling . . . 14
3.5.2 Token Passing . . . 15
3.6 DOCSIS: Multiple Access for Cable Internet . . . 15
3.7 Summary and Reinforcement . . . 15
3.7.1 Key Terms . . . 15
3.7.2 Core Relationships . . . 16
3.7.3 Key Insights . . . 16

4 Switched Local Area Networks (Section 6.4) 16
4.1 Link-Layer Addressing and ARP (6.4.1) . . . 16
4.1.1 MAC Addresses: The Physical Identity . . . 16
4.1.2 ARP: Address Resolution Protocol . . . 17
4.1.3 Sending a Packet Off-Subnet . . . 17
4.2 Ethernet (6.4.2) . . . 18
4.2.1 Overview and History . . . 18
4.2.2 Ethernet Frame Structure . . . 18
4.2.3 Service Model . . . 18
4.3 Link-Layer Switches (6.4.3) . . . 19
4.3.1 Definition and Purpose . . . 19
4.3.2 Switch Table . . . 19
4.3.3 Self-Learning Algorithm (Plug-and-Play) . . . 19
4.3.4 Switches vs. Routers . . . 19

4.4 Virtual Local Area Networks (VLANs) (6.4.4) . . . 20
4.4.1 Motivation . . . 20
4.4.2 Port-Based VLANs . . . 20
4.4.3 VLAN Trunking (802.1Q) . . . 20
4.5 Summary and Reinforcement . . . 21
4.5.1 Key Terms . . . 21
4.5.2 Core Relationships . . . 21
4.5.3 Key Insights . . . 21

5 Link Virtualization: A Network as a Link Layer (Section 6.5) 21
5.1 The Evolution of the "Link" Abstraction . . . 21
5.1.1 Concept Overview . . . 21
5.1.2 The Dial-Up Analogy . . . 22
5.2 Multiprotocol Label Switching (MPLS) . . . 22
5.2.1 Definition and Purpose . . . 22
5.2.2 The MPLS Header . . . 22
5.2.3 Mechanism: Label Switched Routers (LSRs) . . . 23
5.3 Why MPLS? (The Value Proposition) . . . 23
5.3.1 Traffic Engineering . . . 23
5.3.2 Fast Reroute (Restoration) . . . 24
5.3.3 Virtual Private Networks (VPNs) . . . 24
5.4 Summary and Reinforcement . . . 24
5.4.1 Key Terms and Definitions . . . 24
5.4.2 Core Relationships . . . 24
5.4.3 Key Insights / Takeaways . . . 24

6 Data Center Networking (Section 6.6) 25
6.1 Concept Overview . . . 25
6.2 Technical Mechanism: Data Center Architectures . . . 26
6.2.1 A. Hierarchical/Tiered Network Topology . . . 26
6.2.2 B. The Challenge of Limited Host-to-Host Capacity . . . 26
6.2.3 C. Load Balancing . . . 27
6.3 Practical and Intuitive View: Trends and Evolution . . . 27
6.3.1 A. Centralized SDN Control and Management . . . 27
6.3.2 B. Virtualization and Network Layer Abstraction . . . 28
6.3.3 C. The Unique Challenge of Data Center Congestion Control . . . 28
6.4 Summary and Reinforcement . . . 28
6.4.1 Key Terms and Definitions . . . 28
6.4.2 Core Relationships . . . 29
6.4.3 Key Insights / Takeaways . . . 29

7 Retrospective: A Day in the Life of a Web Page Request (Section 6.7) 29
7.1 Concept Overview: The Synthesis of the Protocol Stack . . . 29
7.2 Phase I: Initial Configuration and Name Resolution (L7, L4, L3, L2) . . . 30
7.2.1 Initial Network Configuration via DHCP . . . 30

7.2.2 DNS Query: Resolving the Web Server's IP 30
7.3 Phase II: Establishing Link-Layer Connectivity (L2) 31
7.3.1 ARP: Resolving the Router's MAC Address 31
7.4 Phase III: Request and Response (L7, L4, L3, L2) 31
7.4.1 DNS Resolution and the TCP Handshake 31
7.4.2 HTTP Transaction and Data Transfer 32
7.4.3 Final Delivery 32
7.5 Summary and Reinforcement 33
7.5.1 Key Terms and Definitions 33
7.5.2 Core Relationships 33
7.5.3 Key Insights / Takeaways 33

Chapter 6 - Link Layer

1 Introduction to the Link Layer (Section 6.1)

1.1 Conceptual Framework: The Node-to-Node Perspective

1.1.1 Core Terminology and Abstraction

While the network layer (IP) is responsible for routing a datagram from the *source* host to the *final destination*, the link layer is concerned with a more localized task: moving that datagram across a **single physical link** between two adjacent nodes.

To describe link-layer behavior precisely, we introduce several key terms:

- **Nodes**: Devices that implement link-layer protocols (hosts, routers, switches, WiFi access points).
- **Links**: Communication channels connecting adjacent nodes. These may be wired (copper, fiber) or wireless (WiFi, satellite).
- **Frame**: The link-layer packet. A network-layer datagram is encapsulated into a frame before transmission.

Heterogeneity of Links. A fundamental concept is that as a datagram travels end-to-end, it typically traverses *multiple, heterogeneous link types*. Each individual link may provide different service guarantees.

- A datagram might begin its journey on an Ethernet LAN, traverse a fiber backbone, and conclude on a WiFi hop.
- Each link-layer protocol independently determines its own reliability and error-handling procedures.

The implication is that link-layer protocols are *not uniform* across the network path; each operates autonomously on its specific link.

1.2 The Transportation Analogy

To develop intuition for the relationship between the network and link layers, consider a tourist traveling from Princeton, NJ to Lausanne, Switzerland.

Scenario Overview. A tourist (representing the **datagram**) wishes to travel internationally. A travel agent (the **routing protocol**) plans the itinerary:

1. Limousine from Princeton to JFK Airport
2. Plane from JFK to Geneva
3. Train from Geneva to Lausanne

Mapping to Networking Concepts.

Transportation Analogy	Networking Concept	Function
Tourist	Datagram	Payload being transported
Travel Agent	Routing Protocol	Determines end-to-end path
Leg of Trip	Link	Adjacent-node physical connectio
Transportation Mode (Limo, Plane, Train)	Link-Layer Protocol	Controls per-link transmission b ior

Key Insight: Just as the tourist is successively encapsulated within different transportation modes, the datagram is encapsulated in different link-layer frames as it moves across different links. Each protocol operates independently on its own "leg" of the journey.

1.3 Services Provided by the Link Layer

The link layer supplies a set of services to the network layer. The specific services depend on the protocol used on a particular link.

1.3.1 Framing

- **Definition**: Encapsulation of a datagram inside a link-layer frame.
- **Mechanism**: A header and (often) a trailer are added:

$$\text{Frame} = [\text{Header} \mid \text{Datagram} \mid \text{Trailer}]$$

- **Intuition**: Similar to enclosing a letter in an envelope with a standardized address format.

1.3.2 Link Access (Medium Access Control, MAC)

- **Definition**: Rules governing which node transmits on a shared medium.
- **Point-to-Point Links**: Simple protocols where one sender communicates with one receiver.
- **Broadcast Links**: Require **MAC protocols** (e.g., Ethernet CSMA/CD, WiFi CSMA/CA) to coordinate access and reduce collisions.
- **MAC Addressing**: Frames contain physical (MAC) addresses which are distinct from IP addresses and identify specific network interfaces.

1.3.3 Reliable Delivery

- **Definition**: Ensuring a datagram is delivered across a link without errors.
- **Mechanism**: Utilizes ARQ (ACKs and retransmissions).
- **Graduate-Level Trade-Off**: Even though TCP provides end-to-end reliability, local per-link reliability is useful on *high-error links* (e.g., wireless). Local retransmission avoids expensive global retransmission.
- **Optimization**: Links with very low bit-error rates (e.g., fiber) typically omit reliable delivery to avoid unnecessary overhead.

1.3.4 Error Detection and Correction

- **Definition**: Detecting or correcting bit-level corruption.
- **Mechanisms**:
  - *Error Detection*: CRC, checksums in the trailer.
  - *Error Correction*: Forward Error Correction (FEC) enables correction without retransmission.
- **Implementation**: Usually hardware-accelerated for speed.

1.4 Implementation of the Link Layer

1.4.1 Location: The Network Adapter (NIC)

Most link-layer functionality is implemented in specialized hardware known as the **Network Interface Controller (NIC)**.

Host Architecture Overview.

```
      [ Host Architecture ]
               |
[ CPU / Memory / OS (Software) ]
      ^        |
      | (Bus)  |
      v        v
[ Network Adapter (NIC) ]
      |  - Link-layer controller chip
      |  - Local RAM buffer
      |  - DSP / modem components
      |
      v
[ Physical Link ]
```

1.4.2 Hardware–Software Interface

- **Hardware (Controller)**: Performs framing, error detection, MAC logic, and timing-sensitive operations at line rate (10 Gbps–100 Gbps).
- **Software (Driver)**:
  1. Prepares link-layer addressing information.
  2. Activates and manages NIC hardware.
  3. Handles interrupts to pass received datagrams to upper layers.

1.4.3 Data Flow

Sending Side.

1. OS passes the datagram to the NIC.
2. NIC encapsulates it in a frame, computes error-check bits.
3. Bits are transmitted onto the physical medium.

Receiving Side.

1. NIC receives raw bits and reconstructs the frame.
2. Performs error detection and discards or accepts the frame.
3. If valid, NIC extracts the datagram and interrupts the OS to pass it upward.

1.5 Summary and Reinforcement

Key Terms.

- **Frame**: Link-layer protocol data unit.
- **Node**: Device implementing link-layer logic.
- **MAC Protocol**: Rules for shared-medium access.
- **NIC**: Hardware interface implementing link-layer operations.

Core Relationships.

- **Link vs. Network Layer**: Link layer = node-to-node; network layer = end-to-end.
- **Software vs. Hardware**: Link-layer boundary marks the shift from OS software to NIC hardware.

Key Takeaways.

- Link-layer protocols are independent across different links.
- Services vary: wireless often requires reliability; fiber typically does not.
- High-speed hardware implementations (ASICs, FPGAs) are required for error checking and MAC functions.

2 Error-Detection and -Correction Techniques (Section 6.2)

2.1 The Fundamental Problem: Bit Errors

2.1.1 Concept Overview

At the physical layer, data is transmitted as analog signals that are vulnerable to **attenuation**, **thermal noise**, and **electromagnetic interference (EMI)**. These disturbances may cause the receiver to misinterpret a transmitted bit, such as reading a "0" as a "1".

The Link Layer must therefore provide mechanisms to *detect* and, in some cases, *correct* these bit errors locally, rather than forcing upper layers such as IP or TCP to repeatedly handle corrupted payloads. This is achieved by introducing **redundancy** into the transmitted frame.

2.1.2 The Error-Detection Model

Let:

- D: Data protected by error checking, may include header fields
- EDC: Error-detection/-correction bits (redundancy).

The sender transmits the bit pattern:

$$[\,D \mid EDC\,].$$

The receiver obtains:

$$[\,D' \mid EDC'\,],$$

where D' and EDC' may differ from D and EDC due to bit flips.

The receiver computes a function $f(D')$ and compares it with EDC'.

- If they match: accept the frame.
- If they differ: an error is detected.

Error-Detection Scenario (Conceptual Diagram)

```
   Sender                                          Receiver
Datagram D                                     Datagram D'
     |                                               ^
     v                                               |
[Compute EDC]                                [Check for Errors]
     |                                               ^
     v                                               |
[Frame: D | EDC] --(Bit Errors on Link)--> [Frame: D' | EDC']
```

2.1.3 Key Limitation: Undetected Errors

No error-detection method is perfect. There is always a small probability that bit flips occur in such a way that the corrupted data D' produces a check value matching the corrupted EDC'. Such cases are called **undetected errors**.

2.2 Parity Checks

2.2.1 Single-Bit Parity

Mechanism.

- **Even parity**: append a bit so total # of 1s is even.
- **Odd parity**: append a bit so total # of 1s is odd.

Detection Capability.

- Detects **any odd number** of bit flips.
- Fails for **even-numbered** bit flips (e.g., 2 flipped bits).

Intuition. Simple and cheap but weak, especially for *burst errors* where multiple adjacent bits corrupt.

2.2.2 Two-Dimensional Parity

Mechanism.

- Organize data D into an $i \times j$ bit matrix.
- Compute parity for each row and each column.
- Add a possible overall parity bit.

Correction Capability. A single flipped bit causes a parity error in:

- its **row**, and
- its **column**.

The intersection identifies the erroneous bit, enabling **Forward Error Correction (FEC)** by flipping it back.

Two-Dimensional Parity Illustration

```
          Data Columns         Row Parity
       +-------------------+
Row 1: | 1 0 1 0 1 | 1 |
Row 2: | 1 0 1 1 1 | 0 |
Row 3: | 0 1 1 1 0 | 1 |
       +-------------------+
Col    | 1 0 1 1 1 | 0 |
Parity
```

If, for example, the bit at (Row 2, Col 2) flips from 1 to 0, both Row 2 and Col 2 show parity errors. **Intersection = correctable bit position**.

2.3 Checksumming Methods

2.3.1 Concept Overview

Checksums are lightweight, software-friendly error-detection methods predominantly used in the Network (IP) and Transport (TCP/UDP) layers, where performance must be balanced with acceptable detection strength.

2.3.2 The Internet Checksum (RFC 1071)

Sender Algorithm.

1. Treat the segment as a sequence of 16-bit integers.
2. Sum them using **1's complement arithmetic** (wraparound carry).
3. Take the 1's complement of the final sum to form the checksum.

Receiver Algorithm.

1. Compute the 1's complement sum of all 16-bit words, including the checksum.
2. If the result is all 1s (i.e., $1111 \ldots 1111$), no errors are detected.
3. Otherwise, an error is detected.

Intuition. 1's complement arithmetic allows uniform behavior across different byte-order conventions.

2.4 Cyclic Redundancy Check (CRC)

2.4.1 Concept Overview

CRC is the dominant error-detection technique in the Link Layer (Ethernet, WiFi). It relies on **polynomial arithmetic modulo 2**. Hardware implementations (e.g., linear feedback shift registers) make CRC extremely fast and robust.

2.4.2 Mathematical Framework

- Treat data D as a polynomial $D(x)$ over GF(2).
- Let $G(x)$ (generator polynomial) be a publicly defined $(r+1)$-bit pattern.
- Goal: append an r-bit residue R so that:

$$\frac{D \cdot 2^r \oplus R}{G} = n, \qquad \text{remainder } 0.$$

2.4.3 Sender Algorithm

1. Append r zeros to D to form $D \cdot 2^r$.
2. Divide $(D \cdot 2^r)$ by G using modulo-2 division.
3. The remainder is the CRC value R.
4. Transmit $[\,D \mid R\,]$.

2.4.4 Receiver Algorithm

1. Receive $[\,D' \mid R'\,]$.
2. Divide the whole bitstring by G.
3. If the remainder is zero, frame is assumed uncorrupted; otherwise, error detected.

2.4.5 Example: Polynomial Division

Let $D = 101110$ and $G = 1001$ (so $r = 3$).

Step 1: Append 3 zeros:

$$101110000.$$

Step 2: Perform modulo-2 division:

```
             101011   <-- Quotient (unused)
          -----------
 1001 ) 101110000
        1001
        ----
         0101
         0000
         ----
          1010
          1001
          ----
           0110
           0000
           ----
            1100
            1001
            ----
             1010
             1001
             ----
              011   <-- Remainder R
```

Result. Sender transmits:

$$[\,101110\ 011\,].$$

2.4.6 Practical Power of CRC

CRC-32 (Ethernet) detects:

- All burst errors of length $< r + 1$.
- Any odd number of bit errors.

Probability of undetected error is typically $< 10^{-12}$ for real-world frames.

2.5 Summary and Reinforcement

2.5.1 Key Terms

- **BER (Bit Error Ratio)**: Probability a bit is received incorrectly.
- **FEC (Forward Error Correction)**: Receiver-based correction without retransmission.
- **Modulo-2 Arithmetic**: XOR-based arithmetic with no carries.
- **Generator Polynomial** $G(x)$: Divisor used for CRC.

2.5.2 Core Relationships

- Parity: simplest but weakest.
- Checksums: software-efficient but moderate detection capability.
- CRC: hardware-efficient, extremely strong detection.

2.5.3 Key Insights

- Reliability is probabilistic; no method guarantees perfect detection.
- FEC reduces latency on noisy or high-latency links (e.g., wireless, satellite).
- CRC hardware uses XOR gates and LFSRs for high-speed computation.

3 Multiple Access Links and Protocols (Section 6.3)

3.1 The Multiple Access Problem

3.1.1 Concept Overview

Up to this point, we have examined *point-to-point* links (e.g., a dedicated wire between two routers). However, many networks rely on a **broadcast link**, where *multiple nodes share a common communication medium.* Examples include:

- Traditional shared-medium Ethernet,
- WiFi (802.11),
- Satellite networks,
- Shared coaxial cable systems.

Cocktail Party Analogy. If multiple people in a room speak at the same time, no one is understood. Similarly, nodes on a broadcast channel must coordinate *who transmits when.*

3.1.2 Collisions

When two or more nodes transmit simultaneously, their signals interfere, producing a **collision**.

- Result: the signal becomes garbled and the frame is lost.
- Consequence: channel bandwidth is wasted during corrupted transmission.

3.1.3 The Multiple Access Protocol

A **Multiple Access Protocol** is a distributed algorithm that determines how nodes share a broadcast channel.

Ideal Characteristics. For a channel of rate R with M nodes, a good protocol should:

1. Provide throughput R when one node transmits and R/M per node at full load,
2. Be decentralized (no master node),
3. Be simple and inexpensive to implement.

3.2 Taxonomy of Multiple Access Protocols

Multiple access protocols fall into three broad families:

1. **Channel Partitioning Protocols**:
   (a) divide channel into sections (time/frequency) and assign exclusively.
   (b) allocate piece to node for exclusive use
2. **Random Access Protocols**: allow nodes to transmit at will; detect/recover from collisions.
   (a) allow nodes to transmit at will; detect/recover from collisions.
3. **Taking-Turns Protocols**:
   (a) nodes transmit in a coordinated sequence.

3.3 Channel Partitioning Protocols

3.3.1 TDMA (Time Division Multiple Access)

Mechanism.

- Channel divided into frames with N fixed time slots.
- Each node transmits only in its assigned slot.
- Slots that're unused go idle

Analysis.

- **Pro**: No collisions; perfect fairness.
- **Con**: Highly inefficient at low load—many slots may be idle.

3.3.2 FDMA (Frequency Division Multiple Access)

Mechanism.

- The total bandwidth R is divided into N frequency bands of width R/N.
- Each node transmits on its dedicated band.
- Unused transmission time in frequency bands go idle.

Analysis. Similar pros and cons as TDMA: high efficiency at heavy load but wasteful at light load.

3.4 Random Access Protocols

Optimized for **bursty** traffic. Nodes transmit at the full channel rate R. If a collision occurs, the node waits for a random delay and retransmits.

3.4.1 Slotted ALOHA

Mechanism.

- Time is divided into fixed-size slots of duration L/R.
- Nodes are synchronized to slot boundaries.
- A node transmits at the *beginning* of the next slot when it has a frame.
- On collision, nodes retransmit with probability p in future slots.

Efficiency Analysis. Let each of N nodes attempt transmission with probability p:

$$\Pr(\text{Node A succeeds}) = p(1-p)^{N-1},$$

$$\Pr(\text{Any node succeeds}) = Np(1-p)^{N-1}.$$

Maximizing with respect to p gives:

$$\text{Maximum Efficiency} = \frac{1}{e} \approx 0.37.$$

Intuition. About 37% of slots carry a successful transmission; the rest are idle or suffer collisions.

3.4.2 Pure ALOHA (Unslotted)

Mechanism. Nodes transmit immediately when a frame is ready—no slot synchronization.

Collision Window. A transmission at time t_0 collides with any transmission in $(t_0 - 1, t_0 + 1)$.

Efficiency.

$$\text{Max Efficiency} = \frac{1}{2e} \approx 0.18.$$

3.4.3 CSMA (Carrier Sense Multiple Access)

Philosophy. "Listen before transmitting."

Mechanism.

1. If the channel is idle, transmit.
2. If the channel is busy, wait until idle.

Why Collisions Still Occur? **Propagation delay**. A node may sense the channel idle because another node's signal has not yet arrived.

3.4.4 CSMA/CD (Carrier Sense Multiple Access with Collision Detection)

Philosophy. "Listen while transmitting."

Mechanism.

1. If channel idle $\rightarrow$ transmit.
2. During transmission, monitor channel energy.
3. If a collision is detected:
   - Abort transmission.
   - Send a 48-bit jam signal.
4. Use **binary exponential backoff**:

$$K \in \{0, 1, \ldots, 2^n - 1\},$$

   where n is the number of collisions so far. Wait $K \cdot 512$ bit-times.

Efficiency.

$$\text{Efficiency} = \frac{1}{1 + 5(d_{prop}/d_{trans})}.$$

Insight. Efficiency increases when:

- propagation delay $\rightarrow 0$,
- frame size $\rightarrow \infty$.

Thus CSMA/CD is most effective on short links with large frames.

3.5 Taking-Turns Protocols

3.5.1 Polling

Mechanism. A master node polls each slave node in turn: "Do you have data to send?"

Trade-offs.

- No collisions.
- Polling overhead and single point of failure.

3.5.2 Token Passing

Mechanism. A token frame circulates among nodes. A node may transmit only when holding the token.

Trade-offs.

- Highly efficient at high load.
- Token recovery is complex if a node crashes while holding it.

3.6 DOCSIS: Multiple Access for Cable Internet

DOCSIS divides communication into downstream and upstream paths:

Downstream (CMTS → Cable Modems). Broadcast channel—no contention.

Upstream (Cable Modems → CMTS). Multiple modems contend for access:

- Uses **TDMA** for actual data slots.
- Uses **Slotted ALOHA** for request messages.

Process.

1. Modems send MAP requests via Slotted ALOHA.
2. The CMTS assigns specific TDMA slots for collision-free data transmission.

3.7 Summary and Reinforcement

3.7.1 Key Terms

- **Collision**: simultaneous transmissions causing interference.
- **Carrier Sense**: checking whether the channel is idle.
- **Exponential Backoff**: dynamic waiting time based on congestion.
- **Propagation Delay**: physical travel time of a signal.

3.7.2 Core Relationships

- **ALOHA**: efficient at low load, collapses at high load.
- **TDMA/FDMA**: inefficient at low load, ideal at high load.
- **CSMA/CD**: robust across medium–high loads.

3.7.3 Key Insights

- No protocol is universally best—performance depends on traffic patterns.
- Longer physical links reduce CSMA/CD efficiency (delayed collision detection).
- Exponential backoff is essential to avoid congestion collapse in random-access systems.

4 Switched Local Area Networks (Section 6.4)

4.1 Link-Layer Addressing and ARP (6.4.1)

4.1.1 MAC Addresses: The Physical Identity

Concept Overview. IP addresses are hierarchical and location-dependent (like a postal address), whereas **MAC addresses** are flat, permanent identifiers (like a fingerprint or Social Security Number). A MAC address is burned into the NIC ROM and remains constant regardless of network location; a device moved from New York to Tokyo receives a new IP address but retains the same MAC address.

Technical Mechanism.

- **Format:** 48 bits (6 bytes), written in hexadecimal (e.g., `1A:2B:3C:4D:5E:6F`).
- **Allocation:** IEEE assigns Organizationally Unique Identifiers (OUIs) for the first 24 bits; vendors assign the remaining 24 bits.
- **Broadcast Address:** `FF:FF:FF:FF:FF:FF`. Frames sent to this address are processed by *all* devices on the LAN.

Why Two Addresses (IP and MAC)?

- **Layering Abstraction:** Network-layer routing must be independent of the underlying link technology. Using MAC-only addressing would require global flat routing tables of size proportional to the number of hosts.
- **Modularity:** Using IP addresses exclusively at the link layer would require every link protocol to understand IP addressing.

4.1.2 ARP: Address Resolution Protocol

The Problem. A sender knows the destination's *IP address* but requires the *MAC address* to send a frame on a LAN. ARP solves:

$$192.168.1.5 \longrightarrow 00:13:49:AE:F2:10.$$

Mechanism: The ARP Protocol.

- **ARP Table:** A cache stored in RAM containing entries of the form:

  $$\langle \text{IP Address}, \text{MAC Address}, \text{TTL} \rangle.$$

  TTL ensures entries expire (commonly after 20 minutes).

- **ARP Query (Broadcast):** If an IP address is unknown, the host broadcasts an ARP request:

  $$\text{Dest MAC} = \texttt{FF:FF:FF:FF:FF:FF}.$$

- **ARP Response (Unicast):** The owner of the requested IP replies directly with its MAC address.

- **Caching:** Sender stores the discovered mapping in its ARP table.

ARP Operation Diagram.

```
      [Node A]                                          [Node B]
 (IP: A.A.A.A, MAC: 11-11)                       (IP: B.B.B.B, MAC: 22-22)
       |                                                  |
       |  1. Check ARP Table (Miss)                       |
       |-------------------------------------------------->|
       |  2. ARP Query (Broadcast Frame)                  |
       |     [Src: 11-11, Dst: FF-FF, TargetIP: B.B.B.B]  |
       |                                                  |
       |                                                  | 3. B identifies its IP
       |<--------------------------------------------------|
       |  4. ARP Response (Unicast Frame)                 |
       |     [Src: 22-22, Dst: 11-11, MyMAC: 22-22]       |
       |                                                  |
       |  5. Update ARP Table                             |
       v
```

4.1.3 Sending a Packet Off-Subnet

If Node A sends to a destination Node Z on a remote subnet, *Node A never ARPs for Node Z's MAC*. Instead, Node A ARPs for the **First-Hop Router's MAC**.

- **Destination MAC:** Router's interface MAC.

- **Destination IP:** Node Z's ultimate IP address.

The router removes the link-layer header, examines the IP packet, forwards to the next hop, and constructs a new link-layer frame for the next link.

4.2 Ethernet (6.4.2)

4.2.1 Overview and History

Ethernet became the dominant wired LAN technology, surpassing Token Ring and ATM due to:

1. **Simplicity:** No tokens or complex control mechanisms.
2. **Evolution:** Speed increases (10M → 100M → 1G → 10G → 400G) without changing the frame structure.

Topology.

- **Legacy:** Bus topology, coaxial cable, collisions common.
- **Modern:** Star topology, switches, full-duplex, no collisions.

4.2.2 Ethernet Frame Structure

```
+----------+----------+-----------+-----------+----------------+-------+
| Preamble | Dest MAC | Source MAC|   Type    | Data (Payload) |  CRC  |
| (8 Bytes)| (6 Bytes)| (6 Bytes) | (2 Bytes) | (46-1500 Bytes)|(4 B)  |
+----------+----------+-----------+-----------+----------------+-------+
```

Fields.

- **Preamble (8 bytes):** Seven bytes of 10101010 and one byte of 10101011 for receiver clock synchronization.
- **Addresses:** Destination and source MAC addresses.
- **Type:** Identifies higher-layer protocol (e.g., IPv4, IPv6, ARP).
- **Data:** MTU = 1500 bytes; minimum 46 bytes (padding added if needed).
- **CRC:** 32-bit error detection. Frames failing CRC are dropped (Ethernet does not retransmit).

4.2.3 Service Model

- **Connectionless:** No handshake.
- **Unreliable:** No ACKs; corrupted frames are silently discarded.

4.3 Link-Layer Switches (6.4.3)

4.3.1 Definition and Purpose

A switch is a transparent Layer-2 packet switch that forwards frames based on MAC addresses. Unlike hubs, switches isolate collision domains and support parallel transmissions.

4.3.2 Switch Table

Each switch maintains a table of entries:

$$\langle \text{MAC Address}, \text{Interface}, \text{Timestamp} \rangle.$$

4.3.3 Self-Learning Algorithm (Plug-and-Play)

1. **Frame Arrives:** On interface x with source S and destination D.
2. **Learn:** Update: "MAC S is on interface x".
3. **Forwarding:**
   - If D known and mapped to interface y:
     - If $x = y$: drop (same segment).
     - If $x \neq y$: forward to y only.
   - If D unknown: **flood** to all interfaces except x.

Self-Learning Sequence.

```
Initial State: Empty Table []

Event 1: A (on Port 1) sends to B (on Port 2).
   -> Switch Learns: {A: Port 1}
   -> Switch Unknown Dest B: Floods to Ports 2, 3, 4.

Event 2: B replies to A.
   -> Switch Learns: {B: Port 2}
   -> Switch Checks Dest A: Known! (Port 1)
   -> Switch Forwards ONLY to Port 1.
```

4.3.4 Switches vs. Routers

Feature	Hub	Switch (L2)	Router (L3)
Traffic Isolation	No	Yes (collision domains)	Yes (broadcast domains)
Addressing	None	MAC (flat)	IP (hierarchical)
Routing	None	Self-learning	Routing protocols (OSPF, BGP)
Processing	Bit-level repeat	Frame inspection	Packet inspection + TTL
Throughput	Shared	High (parallel)	Lower (complex processing)

Key Insight. Switches scale well and are plug-and-play, but routers are essential to prevent broadcast storms and implement hierarchical routing.

4.4 Virtual Local Area Networks (VLANs) (6.4.4)

4.4.1 Motivation

In a traditional LAN:

- Excessive broadcast traffic wastes CPU cycles,
- Reorganization requires physical rewiring,
- All users can see broadcast frames (security issue).

4.4.2 Port-Based VLANs

A VLAN-capable switch assigns ports to logical groups.

- Ports 1–8 → VLAN 10 (Engineering).
- Ports 9–16 → VLAN 20 (Sales).

Broadcasts remain within VLAN boundaries. Routing is required for communication across VLANs.

4.4.3 VLAN Trunking (802.1Q)

When VLANs span multiple switches, a trunk link carries frames for all VLANs using the **802.1Q tag**.

802.1Q Frame Tag.

- **TPID:** `0x8100` (identifies tagged frames).
- **TCI:** Contains 12-bit VLAN ID.

Operation:

1. Switch A receives an untagged frame from VLAN 10.
2. Switch A inserts 802.1Q tag and forwards over trunk.
3. Switch B reads tag (ID=10), strips it, and forwards to VLAN 10 ports.

802.1Q Tag Illustration.

```
Original: [Dest][Src][Type][Data][CRC]
Tagged:   [Dest][Src][Type 8100][VLAN ID][Type][Data][CRC]
                     ^-- Added 4 Bytes --^
```

4.5 Summary and Reinforcement

4.5.1 Key Terms

- **MAC Address**: 48-bit hardware identifier.
- **ARP**: Local broadcast-based IP → MAC resolution.
- **Switching Table**: ⟨MAC, Interface, TTL⟩ mapping.
- **Collision Domain**: Range where collisions may occur; broken by switches.
- **Broadcast Domain**: Range where broadcast frames propagate; broken by routers.
- **VLAN**: Logical segmentation of Layer-2 networks.

4.5.2 Core Relationships

- IP gets packets to the correct subnet; MAC gets packets to the correct interface.
- Switches isolate collisions; hubs do not.
- VLANs separate groups logically; routers interconnect them.

4.5.3 Key Insights

- **Self-learning** allowed Ethernet to scale without manual configuration.
- **Plug-and-play:** Ethernet works instantly compared to IP, which requires configuration (DHCP/manual).
- **Scalability Limit:** Broadcast traffic grows with N; routers are required to partition large L2 domains.

5 Link Virtualization: A Network as a Link Layer (Section 6.5)

5.1 The Evolution of the "Link" Abstraction

5.1.1 Concept Overview

Throughout this course, our definition of a "link" has expanded from a simple physical connection to a sophisticated logical abstraction. This evolution culminates in the concept of **Link Virtualization**.

- **Phase 1: The Physical Wire** – A copper wire directly connecting two hosts.
- **Phase 2: The Shared Medium** – A medium shared by many nodes (Ethernet bus, WiFi spectrum) requiring multiple access protocols.

- **Phase 3: The Switched Fabric** – A switched LAN acting as a transparent fabric connecting hosts.
- **Phase 4: The Network as a Link (Virtualization)** – An entire network (e.g., the PSTN or MPLS core) can act as a *single logical link* to the upper layers.

5.1.2 The Dial-Up Analogy

A home user connecting to an ISP via dial-up experiences:

- **Physical Reality:** The signal traverses the PSTN (switches, copper, fiber, multiplexers).
- **Link Layer View:** The operating system sees a simple point-to-point link to the ISP.
- **Virtualization:** The PSTN *virtualizes* a link for the Internet.

5.2 Multiprotocol Label Switching (MPLS)

5.2.1 Definition and Purpose

Multiprotocol Label Switching (MPLS) is a forwarding technology using short, fixed-length **labels** rather than destination IP address lookups.

- **Original Goal:** Speed up packet forwarding (IP lookup was slow in 1990s hardware).
- **Modern Use:** Traffic Engineering (TE), Virtual Private Networks (VPNs), Fast Reroute.
- **Hybrid Nature:** MPLS sits between Layers 2 and 3 ("Layer 2.5").

5.2.2 The MPLS Header

A new 32-bit MPLS header is inserted between the link-layer header (e.g., Ethernet) and the IP header.

```
+------------------+-------------+-----------+----------------------+
| Layer 2 Header   | MPLS Header | IP Header |      Payload         |
| (e.g., Ethernet) | (32 bits)   |           |                      |
+------------------+-------------+-----------+----------------------+
```

Header Fields (32 bits):

1. **Label (20 bits)** – Used to forward packets.
2. **Exp (3 bits)** – Typically used for QoS / Class of Service.
3. **S (1 bit)** – Bottom-of-stack indicator (MPLS supports label stacking).
4. **TTL (8 bits)** – Prevents forwarding loops (same function as IP TTL).

5.2.3 Mechanism: Label Switched Routers (LSRs)

Routers supporting MPLS are called **Label Switched Routers**. They forward based solely on the label rather than inspecting destination IP addresses.

MPLS Forwarding Table: Traditional IP forwarding table:

$$\text{Dest IP Prefix} \rightarrow \text{Out Interface}.$$

MPLS table:

$$[\text{In Label}, \text{In Interface}] \rightarrow [\text{Out Label}, \text{Out Interface}].$$

Forwarding Process:

1. **Ingress (Label Push):** The ingress router assigns a label (e.g., 10) based on the IP header and attaches the MPLS header.
2. **Transit (Label Swap):** A transit LSR swaps the incoming label with the appropriate outgoing label.
3. **Egress (Label Pop):** The final router strips the label and forwards the remaining IP packet normally.

MPLS Forwarding Example.

```
     [R1: Ingress]          [R2: Transit]          [R3: Egress]
      (IP Router)            (LSR)                  (IP Router)
          |                      |                      |
[IP Packet] ---> [ PUSH Label 6 ] ---> [ SWAP 6->12 ] ---> [ POP Label ] -> [IP Packet]
          |       [ Label: 6 ]   |      [ Label: 12 ]   |
          |                      |                      |
     [Lookup Dest IP]       [Lookup Label 6]       [Lookup Dest IP]
```

5.3 Why MPLS? (The Value Proposition)

If routers can now forward IP packets at wire speed, why use MPLS?

5.3.1 Traffic Engineering

- **IP Limitation:** IP routing (OSPF/BGP) selects the *shortest path*, even if congested.
- **MPLS Solution:** Operators define explicit traffic paths:
  - Voice traffic (Label 100) via fast path.
  - Bulk email (Label 200) via slower, uncongested path.
- **Mechanism:** Establish a Label Switched Path (LSP) that behaves like a virtual circuit.

5.3.2 Fast Reroute (Restoration)

- MPLS can precompute backups for every path.
- Failover occurs in under 50ms.
- OSPF convergence may take several seconds.

5.3.3 Virtual Private Networks (VPNs)

- ISPs use MPLS to create traffic-isolated logical networks for customers.
- Customer A's labeled packets and Customer B's labeled packets share the same physical infrastructure but remain isolated.
- Each customer views the ISP backbone as a virtual private link between branch offices.

5.4 Summary and Reinforcement

5.4.1 Key Terms and Definitions

- **MPLS (Multiprotocol Label Switching):** Forwarding technology using short labels rather than IP prefixes.
- **LSR (Label Switched Router):** MPLS-capable router performing label swapping.
- **LSP (Label Switched Path):** Predefined forwarding path taken by MPLS labeled packets.
- **Traffic Engineering:** Steering traffic along controlled paths to optimize performance.

5.4.2 Core Relationships

- MPLS is often described as **Layer 2.5**: encapsulated by L2, encapsulating L3.
- IP is simple and stateless but limited; MPLS is more complex but offers flexible routing, TE, VPNs.

5.4.3 Key Insights / Takeaways

- **Decoupling Routing from Forwarding:** MPLS separates *path selection* (at ingress) from *hop-by-hop forwarding* (label swapping).
- **"Link" as a Construct:** MPLS demonstrates that a "link" to IP may represent an entire network of LSRs and forwarding rules.
- **SDN Connection:** Software-Defined Networking generalizes the MPLS concept: forwarding entries are programmed by a controller rather than by distributed signaling (LDP/RSVP-TE).

6 Data Center Networking (Section 6.6)

6.1 Concept Overview

A **Data Center Network (DCN)** is the internal, complex computer network that interconnects the tens to hundreds of thousands of computing devices (hosts) within a massive data center [**?**]. DCNs are the backbone of modern cloud computing and large Internet service platforms like search, streaming video, e-commerce, social media, and large-scale machine learning systems [**?**]. Modern DCNs must simultaneously support high throughput, low latency, failure resilience, and massive scalability.

Purpose and Applications. DCNs support several large-scale services:

- [**?**] **Content Provision:** Serving Web pages, search results, e-mail, and streaming video to external clients.
- [**?**] **Massively Parallel Computing:** Supporting distributed computing tasks such as search index construction, data analytics, and machine learning pipelines.
- [**?**] **Cloud Computing/Services:** Providing IaaS, PaaS, and SaaS for platforms such as AWS, Azure, and Google Cloud.

Underlying Mechanisms (Hosts and Racks).

- [**?**] The fundamental work unit in a DCN is the **host** (or blade), a commodity server with CPU, RAM, and local storage.
- [**?**] Hosts are physically organized into **racks**, typically containing 20–40 hosts.
- Each rack connects upward into the DCN via a **Top-of-Rack (TOR) switch**, which aggregates host traffic and forwards it toward higher-tier switching layers.

Workload Characteristics (From the Textbook). According to Section 6.6, DCN workloads display the following important patterns:

- **Highly parallel applications** generate *shuffle traffic*, where many hosts exchange intermediate data.
- **Data analytics jobs** exhibit *query fan-out*: a single user request may trigger hundreds of internal queries.
- **East-West traffic dominates**: Most DCN traffic flows between hosts *within* a data center rather than between the data center and the external Internet.

These patterns heavily influence DCN architecture and routing strategies.

6.2 Technical Mechanism: Data Center Architectures

The internal network topology is predominantly hierarchical and designed for massive scale and high throughput [**?**]. This structure resembles a **Clos network** (also called a **fat-tree topology**), providing rich multi-path connectivity and high bisection bandwidth [**?**, **?**]. Conceptually, a Clos fabric behaves like one giant non-blocking switch interconnecting all hosts.

6.2.1 A. Hierarchical/Tiered Network Topology

Traditional DCNs adopt a three-tier hierarchy beneath a set of border routers:

Layer	**Element**	**Function**	**Interconnection**
Edge	Hosts (Blades)	Run applications and store data.	Connect to TOR switches (40–100 Gbps Ethernet).
Tier 3 (Access)	TOR Switch	Aggregates host traffic within a rack.	Connects to Tier-2 switches.
Tier 2 (Aggregation)	Tier-2 Switches	Aggregate traffic from dozens of TORs.	Connect to Tier-1/core switches.
Tier 1 (Core)	Core/Tier-1 Switches	Provide fabric-wide interconnection.	Connect to Tier-2 switches and border routers.
Border	Border Routers	Connect the DCN to ISPs and the public Internet.	Connect upward to external networks.

Oversubscription (Important Detail from Textbook). Traditional hierarchical DCNs often suffer from **oversubscription**, meaning:

$$\text{Capacity(host-to-host)} > \text{Capacity(available upward bandwidth)}.$$

Typical oversubscription ratios in early DCNs were 5:1, 10:1, or even 20:1, severely limiting inter-rack throughput.

6.2.2 B. The Challenge of Limited Host-to-Host Capacity

Problem. [**?**] The tiered architecture limits **bisection bandwidth**: the capacity supporting traffic between hosts in different halves of the network. This constraint is most severe when much of the traffic is *cross-rack*, which is typical for distributed applications.

Mechanism of Failure (Bottleneck). [**?**] A TOR switch may serve 40 hosts at 10 Gbps each (400 Gbps total), yet its uplink capacity to Tier 2 might be only 100 Gbps. When many hosts concurrently communicate externally or across racks, the uplink becomes an immediate bottleneck.

Solution: Increased Interconnection & Multi-Path Routing.

- Higher-tier switches are interconnected in mesh-like Clos/Fat-Tree patterns.
- [**?**, **?**] Multi-path routing (especially via ECMP) distributes traffic across many equal-cost paths.
- [**?**] Increased redundancy simultaneously improves reliability and throughput.

Clos/Fat-Tree Benefits (Explicit from Section 6.6).

- Uniform bandwidth between any two racks.
- Rich path diversity tolerates failures without service interruption.
- Scales naturally by adding more switching stages.

6.2.3 C. Load Balancing

Mechanism. Incoming client requests targeting a public service IP address first reach a **Load Balancer** (typically a Layer 4 device).

Function. [**?**] The load balancer distributes traffic across many internal hosts to balance load, reduce latency, and maximize utilization.

Security and Addressing. [**?**] It also performs a NAT-like function, translating the public IP address into internal private host addresses, preventing direct external access to hosts.

6.3 Practical and Intuitive View: Trends and Evolution

6.3.1 A. Centralized SDN Control and Management

Connection to Chapter 5. Unlike typical ISP networks (with per-router control), DCNs are owned and operated by a single organization. This allows for **centralized SDN control** [**?**, **?**].

Mechanism. A logically centralized SDN controller programs flow tables for the DCN's switches. This enables:

- automated multi-path routing,
- dynamic load balancing,
- consistent network-wide security policy enforcement,
- fast failure recovery.

6.3.2 B. Virtualization and Network Layer Abstraction

Problem: VM Mobility. VMs may move across racks for load balancing or hardware maintenance. Traditional IP/MAC addressing binds addresses to physical topology, complicating mobility.

Mechanism: Flat Layer 2 Abstraction. To support seamless migration, the entire DCN is often treated as a **giant flat L2 network**. Instead of ARP floods:

- a directory service maps VM IP → VM MAC → VM location,
- hosts query this directory for updated mappings.

This allows the DCN to behave like one massive Ethernet switch despite being physically multi-tiered.

6.3.3 C. The Unique Challenge of Data Center Congestion Control

Physical Constraints. DCNs operate with extremely high bandwidth (40–100+ Gbps) and very low latency (microseconds) [**?**].

Congestion Problem. Classic TCP reacts only after packet loss, which is too slow at microsecond timescales. By the time loss occurs, queues may be overwhelmed.

Custom Solutions (DCTCP/DCQCN). [**?**, **?**] Modern DCNs use specialized transport protocols such as:

- **DCTCP**: Uses ECN marking rather than loss to detect early congestion.
- **DCQCN**: A more advanced variant for RDMA over Converged Ethernet (RoCE).

These protocols reduce queue buildup, maintain extremely low latency, and improve throughput.

6.4 Summary and Reinforcement

6.4.1 Key Terms and Definitions

- [**?**] **TOR Switch (Top-of-Rack):** The first-tier switch connecting hosts in a rack.
- [**?**] **Blade:** A thin, modular server unit; the fundamental computing element.
- [**?**] **Clos Network/Fat-Tree:** A multi-stage interconnection network providing high bisection bandwidth.
- **Bisection Bandwidth:** Maximum traffic capacity between any two halves of a network.
- [**?**] **Load Balancer:** Component distributing client requests across multiple servers while performing address translation.

6.4.2 Core Relationships

- [?] **Hierarchy and Scale:** The TOR/Tier-2/Tier-1 architecture supports tens of thousands of hosts as a unified system.
- [?] **Capacity vs. Redundancy:** Interconnecting switches via Clos/Fat-Tree fabrics provides path diversity and supports multi-path routing.
- [?] **Congestion and Latency:** Low-latency DCNs require ECN-driven congestion control rather than TCP's loss-based approach.

6.4.3 Key Insights / Takeaways

- **Complexity at the Edge:** Although the underlying DCN architecture is complex, it is engineered so that hosts experience a simple, uniform, high-speed network.
- [?] **Software-Defined Infrastructure:** Centralized SDN control is critical for scale, automation, and global optimization.
- [?, ?] **Networking as a Cost Factor:** Efficient multi-path routing and congestion control directly influence operational cost and performance.

7 Retrospective: A Day in the Life of a Web Page Request (Section 6.7)

In this section, we take an integrated, holistic journey down and up the protocol stack by following a single Web page request from a user's browser. As emphasized in the textbook, even a "simple" request to download a Web page involves *many (many!)* protocols working together across all layers of the Internet architecture. This retrospective synthesizes DHCP, DNS, ARP, IP, TCP, and HTTP into one continuous narrative of encapsulation, addressing, and hop-by-hop forwarding. Figure 6.32 in the textbook depicts the setting: a student, Bob, plugs his laptop into an Ethernet switch at his school and downloads a page such as `www.google.com`. As we will see, there is a great deal happening "under the hood." (A Wireshark lab at the end of the chapter analyzes similar traces.)

7.1 Concept Overview: The Synthesis of the Protocol Stack

Typing a URL into a browser triggers an extended sequence of protocol interactions. The goals of this retrospective are to (i) trace the full lifecycle of a Web request from application layer down to the physical link and back, and (ii) clearly identify the roles played by DHCP, DNS, ARP, IP, TCP, and HTTP.

We assume a client host C wants to fetch a Web page from a remote server S at `www.somecollege.edu`. The host is connected to a local Ethernet switch, which is connected to a first-hop router R1. As in the textbook scenario, the router is connected to an ISP; for example, Comcast may provide the DNS service for the school, so the DNS server resides *not* on the school's LAN but within the ISP network.

7.2 Phase I: Initial Configuration and Name Resolution (L7, L4, L3, L2)

Before C can send any application-layer request, it must acquire network configuration and learn the server's IP address. This involves DHCP and DNS.

7.2.1 Initial Network Configuration via DHCP

If C is newly connected to the LAN, it must obtain an IP address and other parameters through the Dynamic Host Configuration Protocol (DHCP), an application-layer protocol running over UDP.

Four-Step DHCP Process

1. **DHCP Discover (Broadcast):** C broadcasts a DHCP Discover message to `255.255.255.255` with destination MAC `FF:FF:FF:FF:FF:FF`. Because C has no IP address, the Discover is encapsulated in a UDP segment within an IP datagram whose source IP is `0.0.0.0`. This datagram is then carried inside an Ethernet frame and sent into the LAN.

2. **DHCP Offer (Unicast or Broadcast):** A DHCP server receives the broadcast and replies with a DHCP Offer proposing an IP address IP_C and associated configuration.

3. **DHCP Request (Broadcast):** If multiple offers arrive, C picks one and broadcasts a DHCP Request message, specifying the chosen server's offer.

4. **DHCP ACK (Unicast):** The server responds with a DHCP ACK that includes:

   - IP_C (the leased IP address),
   - subnet mask,
   - default gateway (router R1),
   - the IP address of the DNS server (here provided by Comcast in the textbook scenario).

Intuition This process resembles a newcomer entering an office environment and asking an administrator (DHCP) for their desk number, campus mailbox rules (subnet mask), manager (default gateway), and directory services (DNS).

7.2.2 DNS Query: Resolving the Web Server's IP

Before sending an HTTP request, the browser must obtain the server's IP IP_S from the Domain Name System (DNS).

Client-Side Operation

- The browser extracts the hostname from the URL and hands it to the DNS client.
- The DNS client constructs a DNS query message (application layer).
- The query is placed into a UDP segment (transport layer) with destination port 53.
- The UDP segment is encapsulated into an IP datagram (network layer) with destination equal to the DNS server's IP (provided by Comcast in the text's example).

At this point, the datagram must be sent to R1 for forwarding toward the DNS server.

7.3 Phase II: Establishing Link-Layer Connectivity (L2)

7.3.1 ARP: Resolving the Router's MAC Address

To send the DNS query to R1, C must encapsulate the IP datagram in a link-layer frame addressed to R1's MAC address.

Technical Mechanism

1. C checks its ARP cache; if MAC_{R1} is absent, it broadcasts an ARP query frame.
2. The ARP frame uses destination MAC `FF:FF:FF:FF:FF:FF` and asks: "Who has IP_{R1}?"
3. Every adapter on the LAN receives the broadcast; R1 recognizes its IP and replies with an ARP Response containing MAC_{R1}.
4. C updates its ARP cache and can now send frames addressed to MAC_{R1}.

Key Principle ARP highlights the separation between network-layer addressing (IP) and link-layer addressing (MAC). C *never* needs the MAC address of the remote Web server; only the MAC of the next-hop router at each link.

7.4 Phase III: Request and Response (L7, L4, L3, L2)

With DNS and ARP completed, C can resolve the server address and initiate TCP/HTTP.

7.4.1 DNS Resolution and the TCP Handshake

DNS Response The DNS server returns IP_S to C, completing name resolution.

TCP Connection Setup C creates a TCP SYN segment destined for S:

- The SYN is encapsulated in an IP datagram with destination IP_S.
- The IP datagram is encapsulated in a link-layer frame addressed to MAC_{R1}.
- R1 forwards the datagram hop-by-hop across the Internet.
- The server replies with a SYNACK; C responds with an ACK, completing the three-way handshake.

7.4.2 HTTP Transaction and Data Transfer

Technical Mechanism

1. C sends an HTTP `GET` request for `/index.html` over the established TCP connection.
2. S processes the GET and retrieves the HTML file.
3. S sends the file as a stream of TCP segments; TCP handles reliable, in-order delivery and congestion control.
4. Segments are carried in IP datagrams; datagrams are encapsulated in link-layer frames and forwarded across each hop.

Router Encapsulation/De-encapsulation Example Consider an intermediate router R2:

- **Inbound at R2:** R2 receives a link-layer frame, removes the frame header, checks for errors, removes the IP header, inspects the destination address, and consults its forwarding table.
- **Outbound at R2:** R2 decrements the IP TTL, determines the next hop (R3), uses ARP if needed to obtain MAC_{R3}, constructs a new frame with a fresh link-layer header, and transmits it.

$$\text{Frame}_{in} \rightarrow \text{Datagram} \rightarrow \text{Datagram (TTL-1)} \rightarrow \text{Frame}_{out}$$
$$MAC_{in} \rightarrow IP \rightarrow IP \rightarrow MAC_{out}$$

This process is repeated at every router: full link-layer decapsulation and re-encapsulation, while the IP datagram remains mostly unchanged end-to-end.

7.4.3 Final Delivery

The TCP segments arrive at C:

- TCP reassembles the byte stream.
- The application layer (browser) processes the HTML.
- The Web page appears to the user.

7.5 Summary and Reinforcement

7.5.1 Key Terms and Definitions

- **Link-Local Discovery:** Processes such as DHCP and ARP that allow a host to obtain configuration and determine the MAC address of the next hop.
- **Encapsulation/De-encapsulation:** Headers added as data moves down the stack and removed as data moves up; routers decapsulate only to the network layer.
- **Three-Way Handshake:** TCP's SYN–SYNACK–ACK procedure that establishes a reliable connection.
- **IP Datagram vs. Link-Layer Frame:** The IP datagram's addresses remain constant end-to-end; link-layer frames change at every hop.

7.5.2 Core Relationships

- **Interdependence of Layers:** Application-layer protocols depend on transport-layer services, which depend on IP routing, which requires link-layer addressing.
- **Broadcast vs. Unicast:** DHCP Discover and ARP queries use broadcast; DNS, TCP, and HTTP use unicast once addressing is resolved.

7.5.3 Key Insights / Takeaways

- **TTL is the Only Changing IP Header Field Hop-by-Hop:** Each router decrements the TTL; otherwise the IP datagram stays intact.
- **Layering Abstraction Works:** HTTP is oblivious to whether frames travel over Ethernet, Wi-Fi, MPLS, or a mixture.
- **Initial Setup Dominates Complexity:** DHCP, ARP, and DNS often require more protocol exchanges than the ensuing HTTP data transfer.
- X
    1. Y